Farmly: The Digital Infrastructure for Modern Farming

B. E. Project Report

Submitted to

University of Mumbai

In partial fulfillment of the requirements for the degree of

Bachelor of Engineering
in

Computer Science & Engineering (Data Science)

by

Aafia Shaikh (24DDS04)
Talha Siddiqui (23DS60)

Supervisor
Prof. Shahegul Afroz



Department of Computer Science & Engineering
(Data Science)
Anjuman-I-Islam's Kalsekar Technical Campus
School of Engineering and Technology
An Autonomous Institute Affiliated to University of Mumbai
Plot No. 2 & 3, Sector -16, Near Thana Naka, Khanda Gaon,
New Panvel, Navi Mumbai. 410206

October 2026

Anjuman-I-Islam's Kalsekar Technical Campus
School of Engineering and Technology
An Autonomous Institute Affiliated to University of Mumbai
Department of Computer Science & Engineering (Data Science)
Khanda Gaon,New Panvel, Navi Mumbai. 410206

Certificate

This is to certify that, the Major Project – I entitled

"Farmly: The Digital Infrastructure for Modern Farming"

is a bonafide work done by

Aafia Shaikh (24DDS04)
Talha Siddiqui (23DS60)

and is submitted in the partial fulfillment of the requirement for the degree of

Bachelor of Engineering

in

Computer Science & Engineering (Data Science)

Submitted to

University of mumbai

Project Guide **(Prof. Shahegul Afroz)**	Project Coordinator **(Prof. Shahegul Afroz)**
Head of Department **(Prof. Zeeshan Khan)**	Dean (SoET) **(Dr. Rajendra Magar)**

Major Project – I Approval

This is to certify that the Project entitled **"Farmly: The Digital Infrastructure for Modern Farming"** is a bonafide work done by **Aafia Shaikh (24DDS04)** and **Talha Siddiqui (23DS60)** under the supervision of **Prof. Shahegul Afroz**. This project is approved in the partial fulfillment of the requirement for the degree of **B.E. in Computer Science & Engineering (Data Science)**.

Internal Examiners :

1. ……

2. ……

External Examiners :

1. ……

2. ……

Date : …/…/……

Place : ……

Declaration

We declare that this written submission represents our ideas and does not involve plagiarism. We have adequately cited and referenced the original sources wherever others' ideas or words have been included. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action against us by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Aafia Shaikh (24DDS04)
Talha Siddiqui (23DS60)

Date : …/…/……

Abstract

Agricultural trade in India is still largely recorded through verbal agreements, paper receipts and disconnected applications. As produce moves from the farmer to distributors and retailers, information about its origin, quantity and certification is lost or cannot be verified. Farmly is a web-based digital infrastructure that gives every crop lot a verifiable digital record that follows the produce through the supply chain.

The system consists of a Next.js web application, an Express.js REST API, a PostgreSQL database on Supabase, document storage on IPFS through Pinata, and a Solidity smart contract that acts as a certificate registry on an Ethereum-compatible blockchain. It supports four roles: farmer, field helper, distributor and retailer. A farmer registers a crop lot and uploads a supporting certificate; the backend computes the SHA-256 hash of the document, stores the document on IPFS and registers the hash on the blockchain. When a distributor buys part of a lot, the purchase is processed atomically, a child batch is created for the purchased quantity and a new transaction certificate is generated that references the farmer's certificate. The same happens when a retailer buys from a distributor, so every product carries an append-only chain of certificates from farm to retail. A QR code on every batch opens a public page showing the product journey without prices or personal information, and verification reports file integrity, blockchain registration, issuer authenticity and certification validity separately. A helper calling module, in which a farmer can call an assigned field helper through masked telephone numbers and give crop details by phone for the helper to review, has been designed and is being implemented in this semester.

The implementation was evaluated with automated tests: 8 smart contract tests, 38 database tests covering partial purchases, idempotent retries, overselling and append-only rules, 11 integration tests against a local blockchain, and IPFS upload and retrieval tests. All of these pass. The complete farmer-to-retailer workflow on the hosted database remains to be executed.

Keywords: Agricultural supply chain, Traceability, Blockchain, Smart contract, IPFS, SHA-256, QR code.

Contents

Abstract **i**

List of Figures **iii**

List of Tables **iii**

1 Introduction **1**

1.1 Overview 1

1.2 Motivation 2

1.3 Objectives 3

1.4 Organization of the report 4

2 Literature Survey **5**

2.1 Survey of Existing System 5

2.2 Limitations of Existing System or Research Gap 7

3 Proposed System **9**

3.1 Problem Statement 9

3.2 System Architecture 10

3.3 Proposed Methodology/Techniques 12

3.4 Experimental Setup 16

3.5 Details of Hardware/Software Requirement 17

4 Planning and System Design **18**

4.1 Schedule for Project 18

4.2 System Design 18

4.3 Implementation Plan for Next Semester 24

5 Results and Discussion **26**
5.1 Implementation Details 26
5.2 Expected Results 31

6 Conclusion and Future Work **34**

References **36**

A Plagiarism Report **38**

B List of Publications **39**

Acknowledgement **40**

List of Figures

3.1 System architecture of Farmly (dashed: helper calling module, in progress) 10
3.2 Certificate chain for a partially purchased batch 13
3.3 Purchase and certificate lifecycle 15

4.1 Project schedule (Gantt chart) 19
4.2 Use case diagram 20
4.3 Data flow diagram – Level 0 21
4.4 Data flow diagram – Level 1 21
4.5 Entity relationship diagram of the core entities 22
4.6 Sequence diagram – distributor purchase and certificate generation 23
4.7 Sequence diagram – QR verification by a consumer 23
4.8 Sequence diagram – helper calling (in progress) 24

5.1 Sign-up page with the four supported roles 27
5.2 Farmer dashboard 27
5.3 Origin certificate upload on the lot page 28
5.4 Marketplace listing with the Inspect action 29
5.5 Inspection view with certificate chain and purchase panel 29
5.6 Certificate verification result with four separate checks 29
5.7 Public traceability page opened from a QR code (mobile view) . 30
5.8 Dashboard in light and dark themes 30

List of Tables

1.1 User roles supported by Farmly 2

2.1 Comparison of existing systems 7

3.1 Certificate verification statuses 13
3.2 Hardware requirements 17
3.3 Software requirements 17

4.1 Data dictionary of the traceability tables 22
4.2 Certificate and traceability API endpoints 24
4.3 Implementation plan for the next semester 25

5.1 Summary of test results 31
5.2 Comparison of Farmly with existing systems 33

Chapter 1

Introduction

Agriculture supports a large share of India's population, yet the records that accompany agricultural produce as it is sold are still largely informal. A typical crop lot passes from a farmer to a collection point, a distributor and a retailer, and at each step the quantity, quality, price and certification of the produce are communicated through phone calls, handwritten receipts or messaging applications. When a consumer or a retailer wants to know where a product came from, or whether a certificate presented by a seller is genuine, there is usually no reliable way to check.

This project, **Farmly**, proposes a digital infrastructure in which every crop lot receives a verifiable digital record. The record begins with the farmer's own certificate and grows each time the produce changes hands, so that the complete journey of the product can be inspected by buyers and verified by consumers through a QR code.

1.1 Overview

Farmly is a web application backed by a REST API, a relational database, decentralised document storage and a public blockchain. The platform serves four types of users, summarised in Table 1.1.

A consumer does not need an account. Scanning the QR code attached to a batch opens a public page that shows the product's journey from the farmer to the latest recorded transfer.

The work reported here covers the core platform and the crop supply chain: user registration and role management, farmer and crop lot registration, a mar-

Table 1.1: User roles supported by Farmly

Role	**Responsibilities in the system**
Farmer	Registers crop lots, uploads the original certificate for each lot, views sales and certificate status.
Field Helper	Registers and assists farmers, records crop lots and supporting documents on a farmer's behalf.
Distributor	Inspects farmer lots and their certificates, purchases full or partial quantities, sells to retailers.
Retailer	Inspects distributor stock and its complete history, purchases, and offers traceable products to consumers.

ketplace, partial purchases with quantity tracking, certificate generation and storage, blockchain registration, certificate verification and QR-based public traceability. A helper calling module, which lets a farmer reach the assigned field helper by telephone, is part of this semester's scope; its design and database tables are complete and its implementation is in progress.

1.2 Motivation

The motivation for this project comes from observations of how agricultural produce is traded and documented:

- The quantity declared by a farmer, the quantity weighed at collection and the quantity finally delivered are frequently recorded on separate pieces of paper, which makes disputes difficult to resolve.
- Certificates such as organic certificates or laboratory reports are shared as photographs or photocopies. A buyer has no simple way to confirm that a document has not been edited after it was issued.
- Once a farmer sells produce, the farmer has no visibility into where it goes. Downstream buyers in turn cannot see the original farm-level information.
- Full price transparency is not desirable either: a retailer should not automatically see the price a distributor paid the farmer. Existing marketplaces tend to expose either too little or too much commercial information.
- Many farmers depend on field helpers or cooperatives to use digital tools, so the system has to support assisted workflows and multiple languages.

These observations led us to design a system where trust comes from verifiable records rather than from the reputation of an intermediary.

1.3 Objectives

The aim of the project is to build a platform on which the origin and movement of agricultural produce can be recorded once and verified by anyone. The main objectives of the work completed in this phase are:

1. To provide secure registration and role-based access for farmers, field helpers, distributors and retailers, with the rules enforced by the server and the database and not only by the user interface.

2. To give each farmer and each crop lot a unique digital identity and to record lots with their crop, variety, quantity, harvest date and supporting documents.

3. To build a marketplace where buyers inspect a lot and its certificates before purchasing, and to support partial purchases without losing track of quantities.

4. To store certificate documents on IPFS and anchor their SHA-256 hashes on a blockchain, so that any later modification of a document can be detected.

5. To generate a new transaction certificate for every transfer, linked to all upstream certificates, forming an append-only certificate chain for each product.

6. To let consumers verify a product's journey through a QR code while keeping prices and personal details private.

7. To provide a clean, responsive interface with working light and dark themes and support for eleven languages (English and ten Indian languages).

8. To let farmers who are not comfortable with a web application call their assigned field helper directly, with phone numbers kept private and every call recorded for follow-up (in progress during this semester).

1.4 Organization of the report

The report is organised as follows. Chapter 2 reviews the literature and existing systems for agricultural marketplaces and supply-chain traceability and identifies the research gap. Chapter 3 describes the proposed system: the problem statement, the system architecture, the methods used for certificate hashing, blockchain registration and quantity tracking, the experimental setup and the hardware and software requirements. Chapter 4 presents the project schedule and the system design, including use case, data flow, entity relationship and sequence diagrams, along with the implementation plan for the next semester. Chapter 5 describes the implementation and discusses the results of testing. Chapter 6 concludes the report and outlines future work.

Chapter 2

Literature Survey

This chapter reviews the literature and existing systems that informed the design of Farmly. It covers electronic agricultural marketplaces, blockchain-based traceability systems for agri-food supply chains, and the underlying technologies of decentralised storage and cryptographic hashing. The chapter ends with the limitations of existing solutions and the research gap that this project addresses. The survey is based on published research and publicly documented systems; no field survey of users was conducted in this phase.

2.1 Survey of Existing System

Electronic agricultural marketplaces

The National Agriculture Market (e-NAM) is a pan-India electronic trading portal launched by the Government of India in 2016 that networks existing Agricultural Produce Market Committee (APMC) mandis to create a unified national market for agricultural commodities [1]. e-NAM provides online bidding, price discovery and payment for commodities traded through registered mandis. Its focus is on the trading event at the mandi; it does not follow a lot after the sale, does not maintain a certificate history for the produce, and does not offer consumers a way to verify origin.

Blockchain-based traceability in agri-food supply chains

Tian [2] proposed an agri-food supply chain traceability system for China that combines RFID and blockchain technology to improve the reliability of infor-

mation about food production, processing, warehousing, distribution and sale. Building on this, Tian [3] described a real-time traceability system based on HACCP, blockchain and the Internet of Things, with the aim of providing information to all supply-chain members with openness and transparency.

Caro et al. [4] presented AgriBlockIoT, a fully decentralised blockchain-based traceability solution for agri-food supply chain management that integrates IoT devices producing and consuming digital data along the chain. They implemented the system on both Ethereum and Hyperledger Sawtooth and compared the two in terms of latency, CPU and network usage.

Salah et al. [5] proposed a blockchain-based approach for soybean traceability that uses Ethereum smart contracts to govern transactions and events among supply-chain participants, and stores large documents on IPFS while keeping only their hashes on the blockchain. This separation of on-chain proofs and off-chain storage is directly relevant to Farmly's design.

Lin et al. [6] proposed a food traceability system for smart agriculture that combines blockchain with IoT data collection, while Leng et al. [7] proposed a double-chain architecture for the agricultural supply chain in which one chain records user information and another records transactions, to address the efficiency and privacy limitations of a single public chain.

Commercial systems also exist. IBM Food Trust, built on Hyperledger Fabric, is a permissioned blockchain network that allows food-supply participants to share traceability data [8]. It is designed for large enterprises and requires membership of the network.

Reviews and enabling technologies

Kamilaris et al. [9] reviewed blockchain initiatives in agriculture and food supply chains and concluded that, although the technology is promising, most projects were at an early stage and faced barriers including the cost of adoption, the lack of standards and the difficulty of linking digital records to physical products. Galvez et al. [10] discussed the future challenges of blockchain for food traceability analysis and noted that the trustworthiness of a blockchain record is limited by the trustworthiness of the data entered into it.

Farmly builds on several enabling technologies. Bitcoin [11] introduced a

public ledger in which records cannot be altered without redoing the proof of work, and Ethereum [12] generalised this into a platform for smart contracts. The InterPlanetary File System (IPFS) [13] is a content-addressed, peer-to-peer file system in which every file is identified by a hash of its content (a CID), so that a stored file cannot be changed without its identifier changing. The SHA-256 algorithm, standardised in FIPS 180-4 [14], produces a 256-bit digest of a document; any change to the document produces a different digest. Finally, the OWASP API Security Top 10 [15] identifies broken object-level and property-level authorisation among the most serious risks for web APIs, which motivated Farmly's server-side enforcement of roles and field-level price visibility.

Table 2.1 summarises the surveyed systems.

Table 2.1: Comparison of existing systems

System	**Approach**	**Strengths**	**Limitations**
e-NAM [1]	Government electronic mandi trading portal	National reach, online price discovery and payment	No lot traceability after sale, no certificate history, no consumer verification
Tian [2, 3]	RFID / IoT with blockchain	End-to-end visibility of food data	Relies on RFID and IoT hardware; certificate documents not addressed
AgriBlockIoT [4]	IoT data on Ethereum and Hyperledger Sawtooth	Fully decentralised; performance compared on two platforms	Focus on sensor data; no marketplace or partial-quantity handling
Salah et al. [5]	Ethereum smart contracts with IPFS	Hashes on-chain, documents on IPFS	Single commodity; no user-facing marketplace or price confidentiality
Leng et al. [7]	Double-chain architecture	Separates user and transaction data	Conceptual architecture; no implemented application
IBM Food Trust [8]	Permissioned Hyperledger Fabric network	Enterprise adoption	Requires network membership; not designed for small farmers

2.2 Limitations of Existing System or Research Gap

From the survey, the following limitations of existing solutions were identified:

1. **Trading and traceability are separate.** Marketplaces such as e-NAM support the sale but not the journey of the produce afterwards, while

traceability research prototypes rarely include a usable marketplace.

2. **Partial quantities are not modelled.** Real lots are split: a distributor buys part of a lot and sells part of that onwards. Most systems record a single product moving along the chain and do not guarantee that the same quantity cannot be sold twice.

3. **Integrity is often over-claimed.** A record on a blockchain proves that data has not changed since it was written, not that the data was true. Few systems distinguish between file integrity, issuer authenticity and the validity of a certification claim when presenting results to users [10].

4. **Commercial confidentiality is ignored.** Systems tend to make all transaction data visible to every participant, whereas downstream buyers should not see upstream prices.

5. **Accessibility for small farmers is limited.** Enterprise solutions require network membership and technical expertise, and research prototypes rarely provide assisted workflows or local-language interfaces.

Farmly addresses this gap by combining a marketplace with an append-only certificate chain that supports partial purchases, by separating the checks performed during verification, by enforcing price visibility on the server, and by providing assisted, multilingual workflows on a standard web platform.

Chapter 3

Proposed System

Farmly is designed as a layered web platform. The user-facing application and the API handle day-to-day work such as registering lots and recording purchases, while two additional layers provide trust: IPFS stores certificate documents in a content-addressed form, and a smart contract on a blockchain stores a permanent fingerprint of every certificate. This chapter describes the problem being solved, the architecture of the system, the techniques used and the setup in which the system was tested.

3.1 Problem Statement

To design and implement a secure, role-based digital platform on which farmers register crop lots with supporting certificates, distributors and retailers can inspect and purchase full or partial quantities of those lots, and every transfer automatically produces a new certificate linked to all earlier certificates, such that:

- the integrity of every certificate document can be verified by recomputing its SHA-256 hash and comparing it with a hash registered on a blockchain;
- quantities are conserved, so that the same quantity of produce can never be sold twice, even when requests are retried;
- the complete journey of a product, from the farmer's original certificate to the latest retail transfer, can be viewed by a consumer through a QR code;

- commercial prices and personal information are visible only to the parties entitled to see them; and
- failures in external services (IPFS or the blockchain) never cause a certificate to be shown as complete or verified when it is not.

The salient features of the proposed system are role-based access for four user types, a unified farmer and crop lot identity, a marketplace with an inspection view, atomic partial purchases with child batches, an append-only certificate bucket for each batch, decentralised document storage, blockchain anchoring, structured verification and a public traceability page.

3.2 System Architecture

Figure 3.1 shows the architecture of the system. It consists of six main components; the telephony component, drawn dashed, belongs to the helper calling module that is in progress.

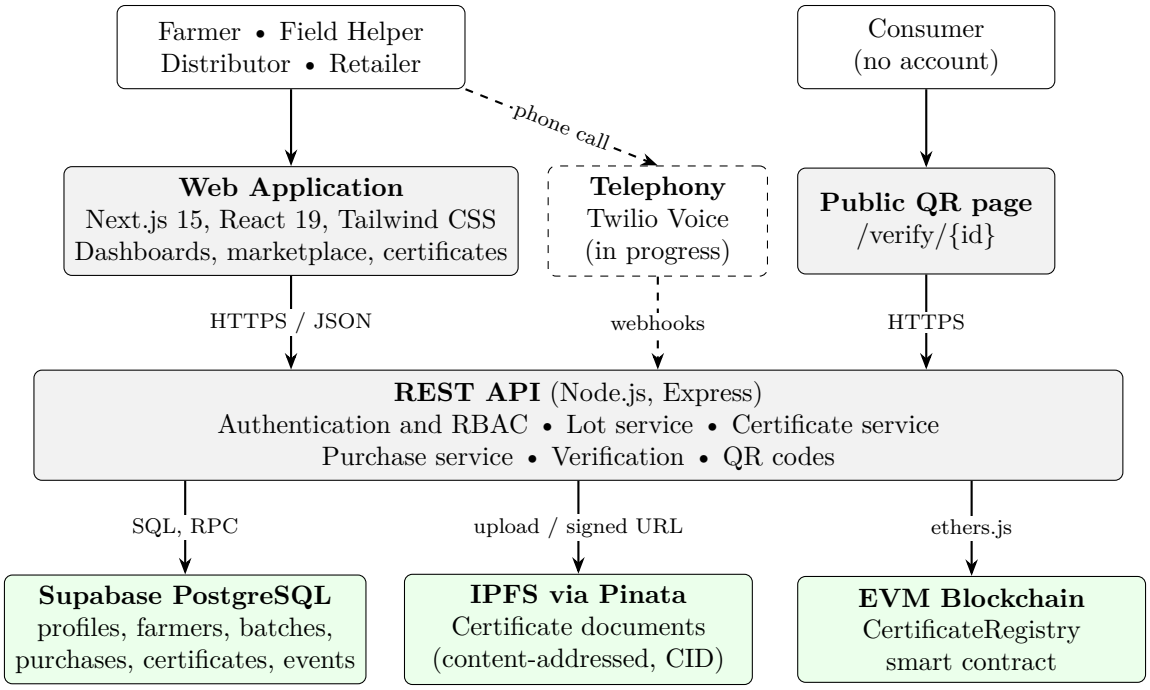


Figure 3.1: System architecture of Farmly (dashed: helper calling module, in progress)

1. **Web application.** Built with Next.js and React, the application provides role-specific dashboards, crop lot registration, the marketplace and

inspection view, certificate upload and verification, and the public QR traceability page. It supports light and dark themes and eleven languages.

2. **REST API.** An Express.js server written in TypeScript authenticates every request, enforces role-based access control, validates all inputs and coordinates the database, IPFS and blockchain. All credentials for external services are held only by this layer.

3. **Database.** A PostgreSQL database hosted on Supabase stores users, farmers, crop lots (batches), purchases, certificates and an audit trail of events. Certificate and purchase tables are protected by row-level security so that only the backend can access them, and database triggers make them append-only.

4. **IPFS storage.** Certificate documents are uploaded through the Pinata service. IPFS identifies every file by a content identifier (CID) derived from its content. Farmer documents are stored privately by default and can only be opened through short-lived signed links.

5. **Blockchain registry.** A Solidity smart contract, `CertificateRegistry`, stores for every certificate the SHA-256 hash of the document, the IPFS CID, the batch and transaction references, the registering account and a timestamp. Registered records cannot be overwritten, and revocations are recorded as separate events.

6. **Telephony (in progress).** The helper calling module connects a farmer's phone call to the assigned field helper through the Twilio Programmable Voice service [21]. The telephony provider notifies the API of call events through webhooks, and call records are stored in the database.

3.3 Proposed Methodology/Techniques

Certificate hashing and blockchain anchoring

For every certificate document D, the backend computes its digest over the exact bytes of the file using SHA-256 [14]:

$$h(D) = \text{SHA-256}(D), \qquad h(D) \in \{0,1\}^{256} \tag{3.1}$$

The document is uploaded to IPFS, which returns its CID. The backend then calls the smart contract's `registerCertificate` function with the certificate identifier, batch identifier, transaction identifier, source certificate identifier, $h(D)$ and the CID. Identifiers are converted to fixed-length 32-byte values using the Keccak-256 function, so that no personal data or free text is written on-chain. A certificate is reported as registered only after the blockchain transaction has been confirmed.

Certificate verification

Verification retrieves the stored document D' (or a file supplied by the user), recomputes its hash and compares it with the hash h_{chain} stored on the blockchain:

$$\text{Integrity}(D') = \begin{cases} \text{MATCH} & \text{if SHA-256}(D') = h_{\text{chain}} \\ \text{MISMATCH} & \text{otherwise} \end{cases} \tag{3.2}$$

Integrity alone does not prove that a certificate is genuine or valid. The system therefore reports four separate checks: file integrity, blockchain registration, issuer authenticity and certification validity (revocation and expiry). These are combined into a single status according to Table 3.1, with the most severe condition taking precedence. A certificate is never shown as verified merely because a CID exists or a transaction was submitted.

Certificate bucket and append-only history

Each batch has a logical certificate bucket: the ordered set of certificates attached to the batch and to all of its ancestors. The bucket begins with the

Table 3.1: Certificate verification statuses

Status	**Meaning**
Verified	Document hash matches the confirmed on-chain hash and all references match.
Issuer Not Verified	The file is intact and registered, but the issuing organisation has not been independently verified.
Mismatch	The document hash or the on-chain references differ from the registered values.
Not Registered	No matching on-chain record exists although one was expected.
Pending	IPFS storage or blockchain registration has not completed yet.
Unavailable	IPFS or the blockchain could not be reached, so the check could not be completed.
Revoked / Expired	The certificate has been revoked, or its expiry date has passed.

farmer's original certificate. Each purchase creates a child batch and a new transaction certificate whose references point to the source certificate and to every upstream certificate, as shown in Figure 3.2. Certificates and purchases can never be deleted or edited; corrections are recorded as new versions that reference the previous one, and revocations are recorded both in the database and on the blockchain while the original proof remains readable.

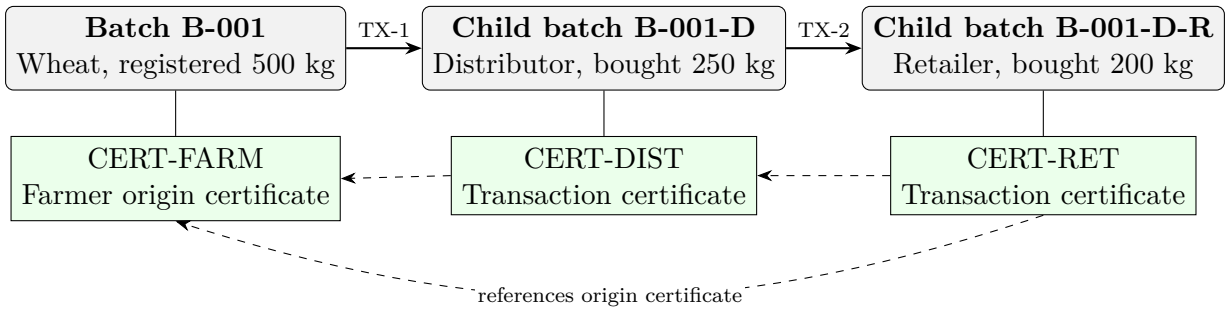


Figure 3.2: Certificate chain for a partially purchased batch

Partial purchases and quantity conservation

Let $q_0(b)$ be the quantity with which batch b was created and $P(b)$ the set of purchases made from it. The available quantity is

$$q_{\text{avail}}(b) = q_0(b) - \sum_{p \in P(b)} q(p), \qquad q_{\text{avail}}(b) \geq 0 \tag{3.3}$$

A purchase is processed by a single database function, `farmly_purchase_batch`, which runs as one transaction. It locks the source batch row (`SELECT … FOR UPDATE`), checks that the buyer's role may buy from the seller's role (distributors buy from farmers, retailers buy from distributors), checks that the requested quantity is available and that the source has an active certificate, deducts the quantity, creates the buyer's child batch and records the purchase. Because the row is locked, two concurrent purchases from the same batch are serialised and cannot together exceed the available quantity. A database constraint additionally guarantees that the available quantity never becomes negative.

Idempotent requests

Network failures may cause a client to resend a purchase request. Every purchase request carries an idempotency key k chosen by the client. The database stores k with a fingerprint f of the request parameters, and the function behaves as

$$\text{Purchase}(k, f) = \begin{cases} \text{existing purchase} & \text{if } k \text{ was used with the same } f \\ \text{rejected} & \text{if } k \text{ was used with a different } f \\ \text{new purchase} & \text{otherwise} \end{cases} \tag{3.4}$$

so a retried request never deducts the quantity twice or creates a duplicate certificate.

Reliable purchase and certificate lifecycle

The database, IPFS and the blockchain cannot be updated in a single transaction, so each purchase moves through explicit states (Figure 3.3). The quantity is reserved first; the transaction certificate is then generated, stored on IPFS and registered on the blockchain. If IPFS or the blockchain is unavailable, the purchase remains reserved for the buyer and the certificate stays in a pending state that can be retried safely. Before resubmitting a registration, the backend checks the blockchain for an existing record so that a transaction that succeeded on-chain but was not saved in the database is reconciled instead of duplicated.

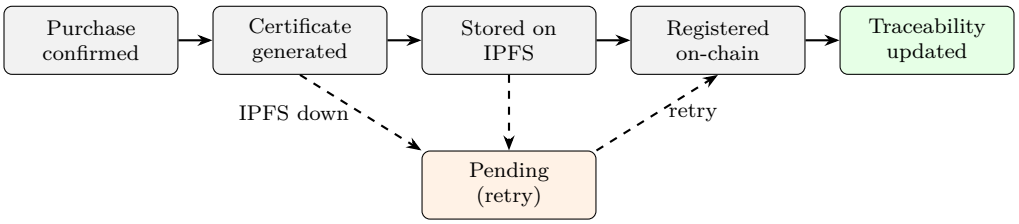


Figure 3.3: Purchase and certificate lifecycle

Role-based access and price confidentiality

Roles are enforced at three levels. The API rejects any registration with a role other than farmer, field helper, distributor or retailer; database triggers prevent clients from assigning or changing a role directly; and every certificate and purchase endpoint checks the caller's role. Prices are stored in a restricted field of each transaction certificate and are returned only to the buyer and seller of that particular transfer. Public pages show only public-safe fields: the farmer's first name and initial, district and state, quantities, dates, hashes and blockchain references.

Helper calling (in progress)

Many farmers are more comfortable speaking to a person than using a web application. The helper calling module lets a farmer reach the assigned field helper by telephone. It supports two flows:

1. **Call a helper.** The farmer calls the Farmly number, or presses *Call my helper* in the application to request a call. The system identifies the farmer from the registered mobile number, looks up the field helper assigned to that farmer, and connects the two through the telephony provider. Neither party sees the other's personal number, because the call is bridged through the provider's number (number masking). If the helper does not answer, a callback request is recorded and appears among the helper's pending tasks.

2. **Crop details by phone.** During the call, the farmer selects a language and describes the crop, variety, quantity and expected harvest date. The call details, transcript and extracted crop data are stored as a submission with status *pending*. The field helper reviews the submission, corrects it if

necessary, and either approves it, which registers a crop lot for the farmer, or rejects it with a reason.

A phone call alone never creates a crop lot or a purchase: every submission must be reviewed by the helper, in line with the rule that helpers assist but do not silently perform critical transactions. Calls are recorded only with the farmer's consent, recordings are accessible only to the assigned helper, and every call is logged with its duration and outcome.

The database tables for this module, `voicegenie_calls` and `voicegenie_submissions`, are already part of the schema. The telephony integration, the webhook endpoints and the helper review screen are being implemented during this semester.

3.4 Experimental Setup

The system was developed and tested on a single development machine. The following test environments were used:

- **Smart contract tests** were run on the in-memory Hardhat network using Mocha and Chai.
- **Database tests** executed the complete base schema and migration on PGlite, a build of PostgreSQL compiled to WebAssembly, with stand-ins for Supabase's authentication schema and roles. The test data followed the scenario used throughout this report: a farmer registers 500 kg of wheat, a distributor buys 250 kg and a retailer buys 200 kg from the distributor.
- **Blockchain integration tests** ran the compiled backend blockchain service against a local Hardhat node (chain ID 31337), after deploying the certificate registry contract to that node.
- **IPFS tests** uploaded small test files to Pinata as public and private files, downloaded them again and compared their SHA-256 hashes.
- **User interface tests** loaded every page of the application for each role in both light and dark themes and measured the contrast ratio of every visible text element against its background, following the WCAG definition

of relative luminance. Pages were also checked at a mobile width of 375 pixels for horizontal overflow.

A test was counted as passed only if the observed behaviour matched the expected behaviour exactly; for example, a purchase exceeding the available quantity was expected to be rejected and the source quantity was expected to remain unchanged.

3.5 Details of Hardware/Software Requirement

Table 3.2 lists the hardware and Table 3.3 the software used to develop and run the system.

Table 3.2: Hardware requirements

Component	**Specification**
Development machine	64-bit processor, 8 GB RAM or more, 10 GB free disk space
Server (deployment)	Any host able to run Node.js 20 or later
Client devices	Desktop, tablet or smartphone with a modern web browser
Network	Internet connection for Supabase, Pinata and the blockchain RPC endpoint

Table 3.3: Software requirements

Category	**Software**
Operating system	Windows 10/11, Linux or macOS
Runtime	Node.js 22, npm
Frontend	Next.js 15, React 19, TypeScript 5, Tailwind CSS 3, React Hook Form, Zod, Lucide icons [16]
Backend	Express 4, TypeScript 5, Zod, Supabase JS client, ethers.js 6 [20], qrcode
Database	PostgreSQL on Supabase, with row-level security [17]
Decentralised storage	IPFS through the Pinata Files API [18]
Blockchain	Solidity 0.8.24, Hardhat 2.22 [19]; local Hardhat network and Ethereum Sepolia testnet
Testing	Mocha and Chai (contracts), PGlite (database), ESLint, TypeScript compiler
Telephony (in progress)	Twilio Programmable Voice for calls and webhooks [21]
Tools	Visual Studio Code, Git, a modern web browser

Chapter 4

Planning and System Design

This chapter presents the schedule followed during this phase of the project and the design of the system using structured analysis and design tools: a use case diagram, data flow diagrams, an entity relationship diagram, sequence diagrams, the database design and the API design. It ends with the plan for the next semester.

4.1 Schedule for Project

The project was planned in phases: requirements and architecture, the core web application, the backend and database, and finally the role, interface and certificate-chain work. The implementation dates in Figure 4.1 are taken from the project's version-control history, which began on 17 September 2026. The requirements and architecture phase preceded the first commit. The helper calling module is scheduled for the remainder of this semester and is shown hatched.

4.2 System Design

Use case diagram

Figure 4.2 shows the main use cases of the system. Farmers and field helpers register farmers and crop lots and upload origin certificates. Distributors and retailers inspect batches and purchase them. Any user, including a consumer without an account, can view the public traceability page and verify a certifi-

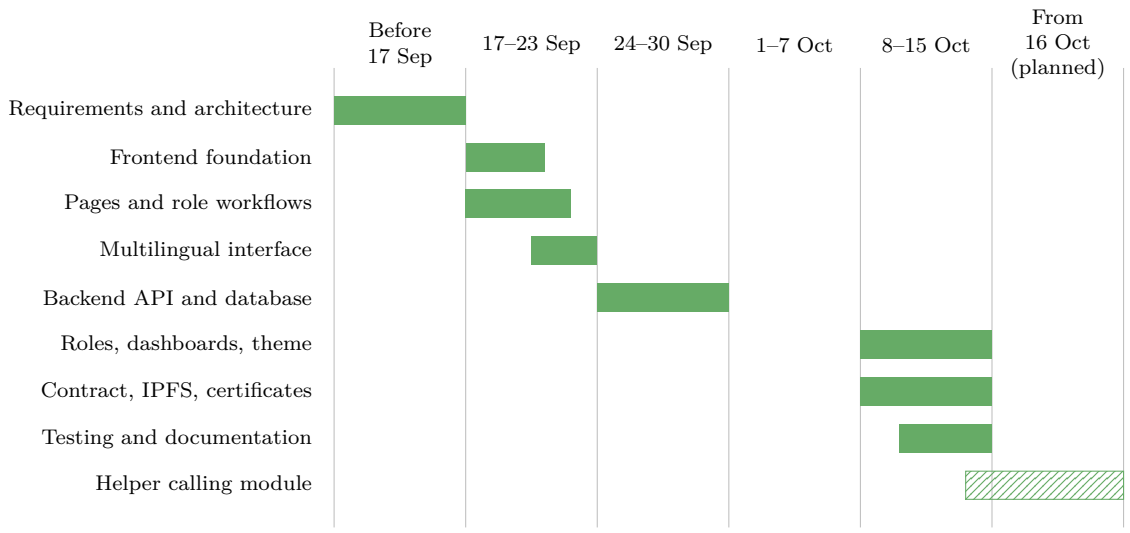


Figure 4.1: Project schedule (Gantt chart)

cate. Calling a field helper, used by farmers and helpers, is part of the module in progress and is shown dashed.

Data flow diagrams

Figure 4.3 is the context-level (Level 0) data flow diagram. It shows Farmly as a single process exchanging data with its users and with the two external services.

Figure 4.4 decomposes the system into its main processes and data stores.

Database design

Figure 4.5 shows the entity relationship diagram of the core entities. A farmer has many batches; a batch may have a parent batch, forming the lineage created by partial purchases; a purchase links a source batch to the child batch it creates; and a certificate belongs to a batch, may be created by a purchase, and references its source certificate.

The base schema contains 28 tables covering users, farmers, crop availability forecasts, batches, evidence, quality assessments, collection arrivals, offers, payments, disputes, notifications and transactions. The traceability migration adds three tables, described in Table 4.1. The helper calling module uses two tables that are already in the base schema: `voicegenie_calls`, which stores

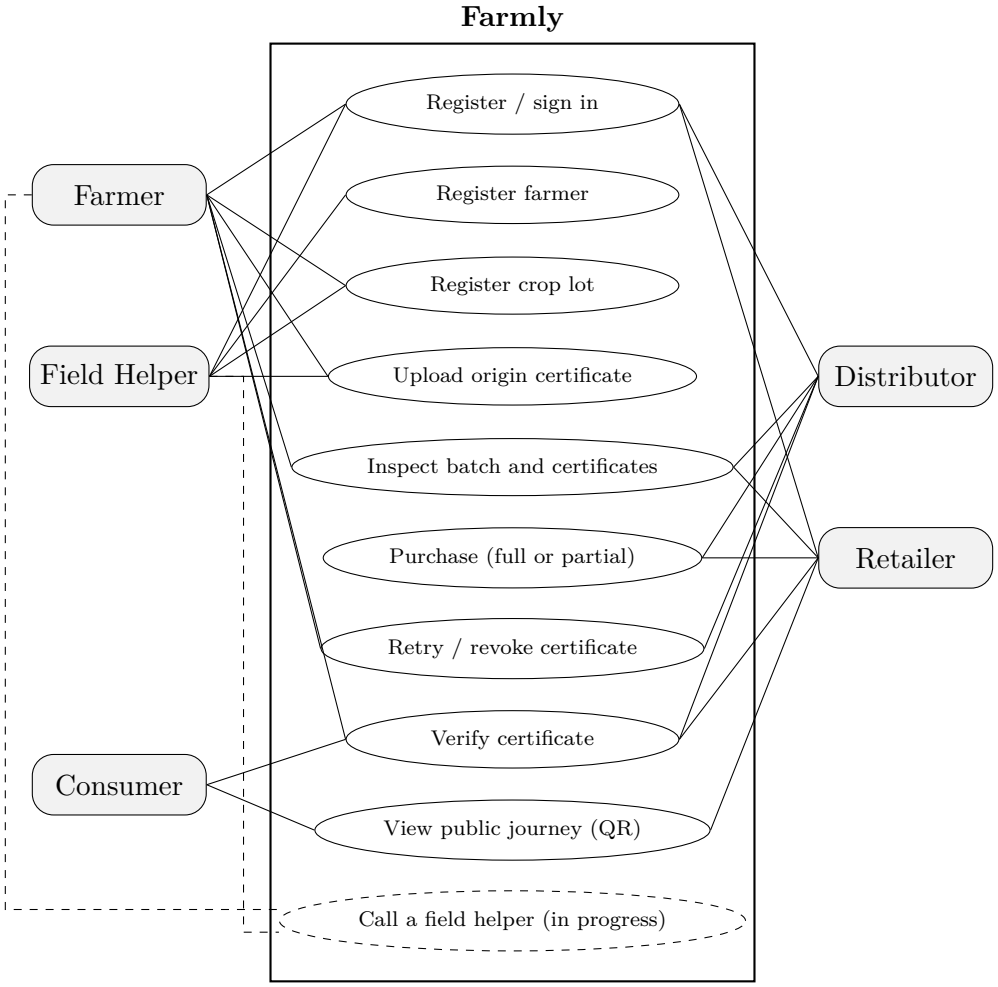


Figure 4.2: Use case diagram

each call's provider identifier, phone number, language, duration, transcript, detected intent and status, and `voicegenie_submissions`, which stores the crop details extracted from a call together with the reviewing helper, the review outcome and the resulting batch.

Sequence diagrams

Figure 4.6 shows the sequence of a distributor purchase. The quantity is reserved in a single database transaction before any external service is called, so a failure in IPFS or the blockchain never loses the purchase.

Figure 4.7 shows how a consumer verifies a product by scanning its QR code.

Figure 4.8 shows the planned sequence of the helper calling module (in progress). The telephony provider bridges the call so that neither party's num-

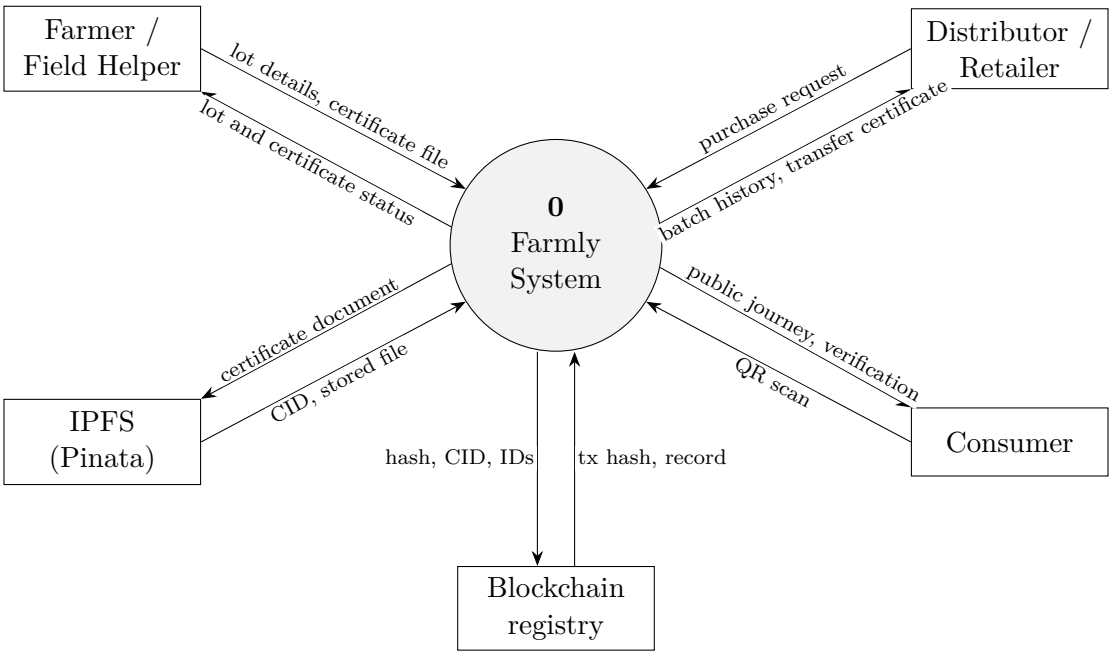


Figure 4.3: Data flow diagram – Level 0

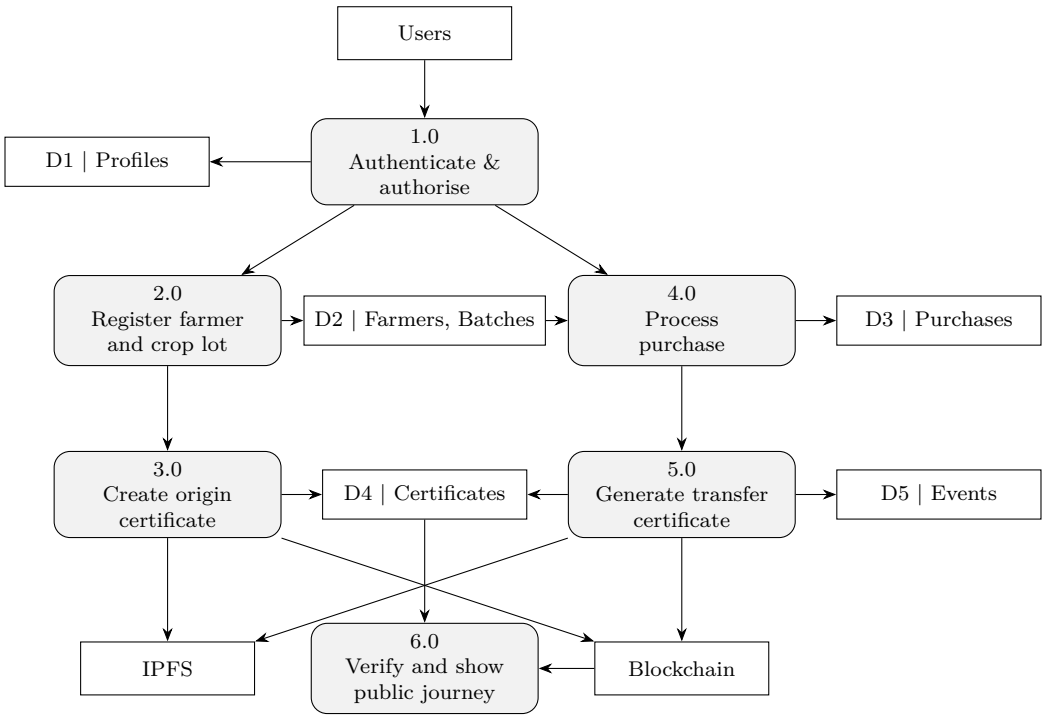


Figure 4.4: Data flow diagram – Level 1

ber is revealed, and crop details given by phone become a pending submission that the helper must review.

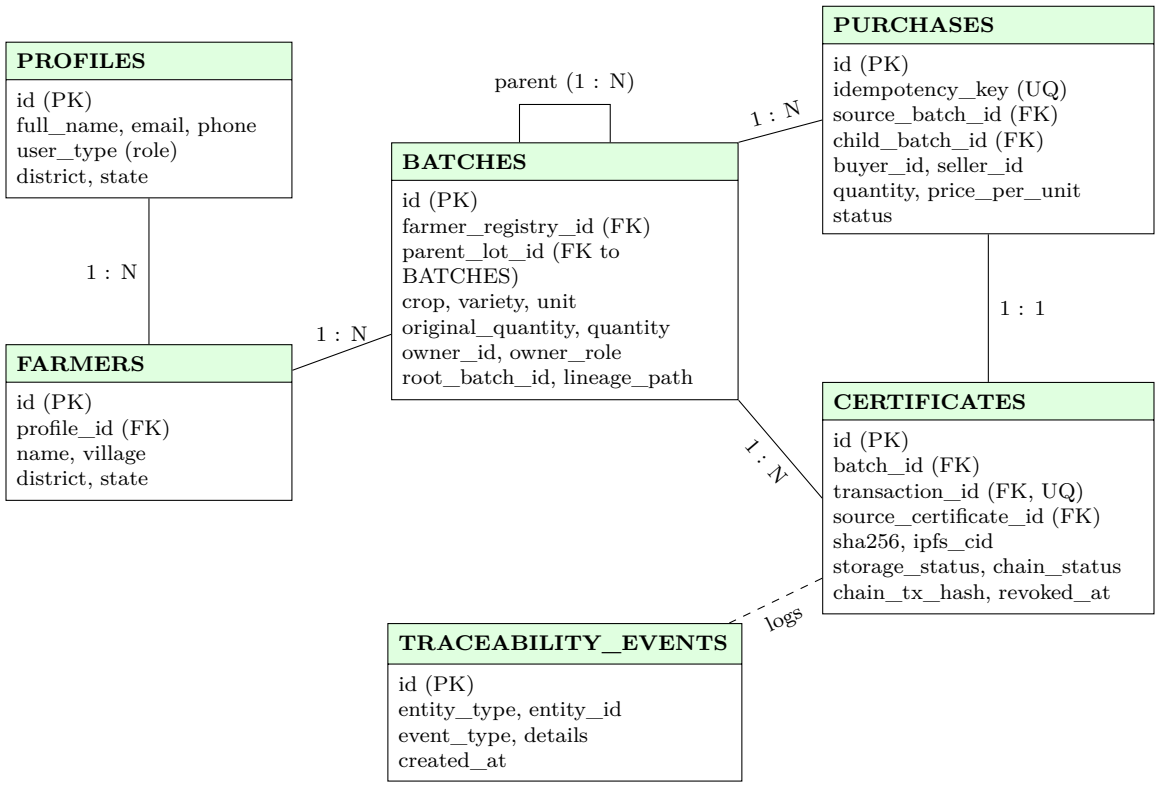


Figure 4.5: Entity relationship diagram of the core entities

Table 4.1: Data dictionary of the traceability tables

Table	**Purpose and key rules**
purchases	One row per transfer. Server-generated transaction ID, unique idempotency key with a request fingerprint, source and child batch, buyer and seller, quantity, private price, lifecycle status. Rows cannot be deleted; quantities and parties cannot be changed.
certificates	One row per certificate: type (farmer origin, distributor or retailer transfer), batch, transaction, source and upstream certificate IDs, version, SHA-256 hash, IPFS CID, storage and blockchain status, revocation. Content is immutable; only status fields can progress.
traceability_events	Append-only audit trail of every step: creation, IPFS storage, blockchain submission and confirmation, failures, retries and revocations.

API design

The REST API exposes 62 route handlers under `/api/v1`. Table 4.2 summarises the endpoints added for the certificate chain.

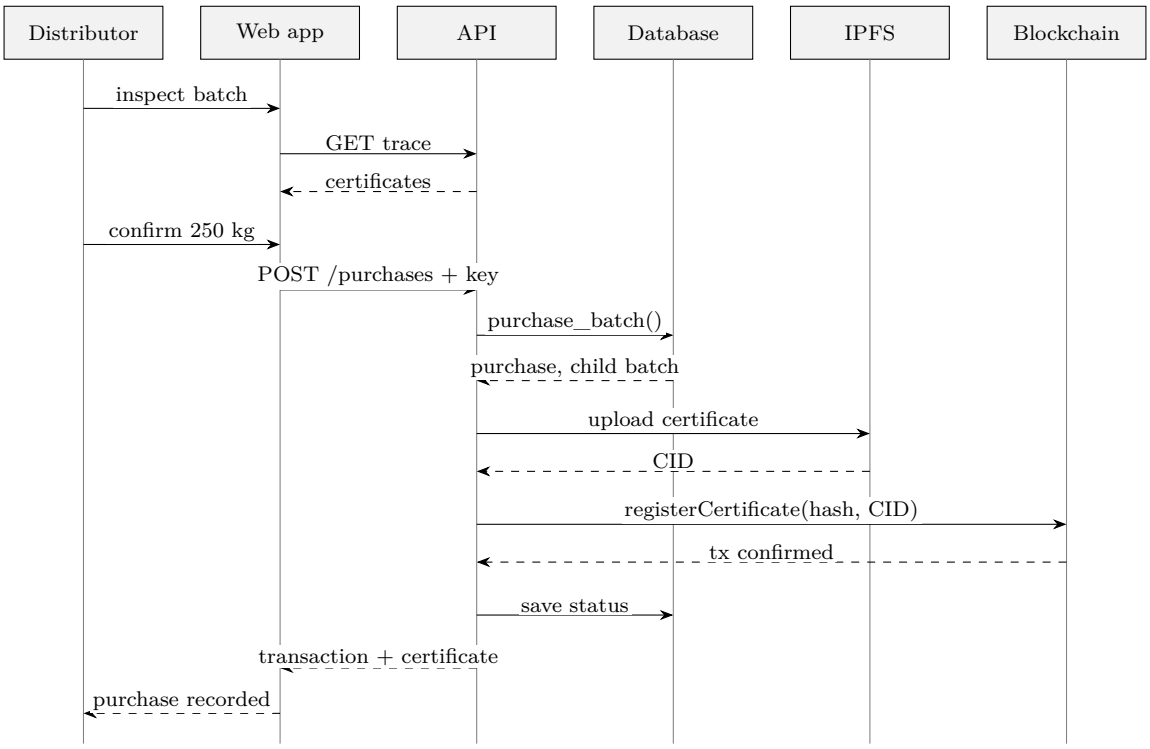


Figure 4.6: Sequence diagram – distributor purchase and certificate generation

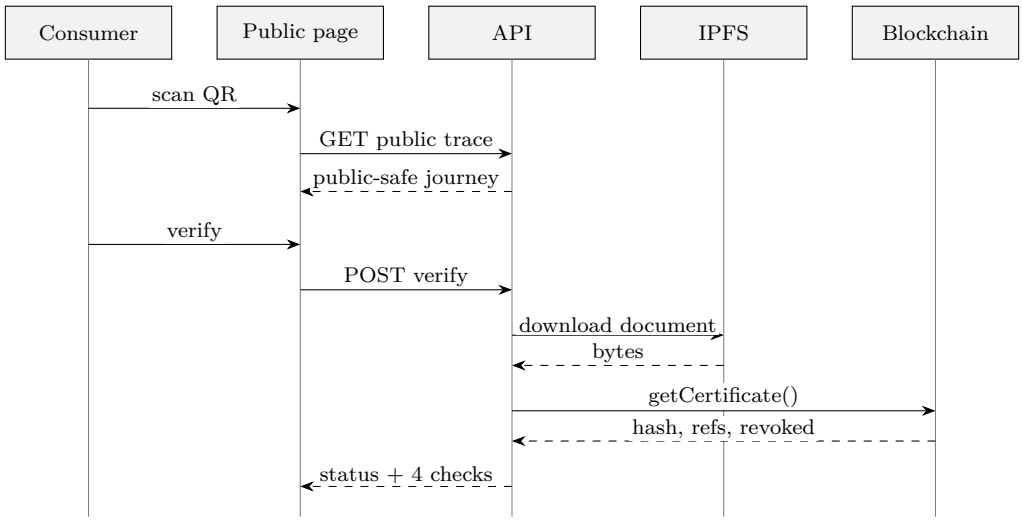


Figure 4.7: Sequence diagram – QR verification by a consumer

User interface design

The interface was designed to be calm and readable: a sidebar with only the items each role needs (for a farmer: Dashboard, My Produce Lots and Marketplace), a top bar with language, theme and notification controls, and dashboards with three summary cards followed by clearly separated lists. Colours are defined as theme variables so that every page works in both light and dark

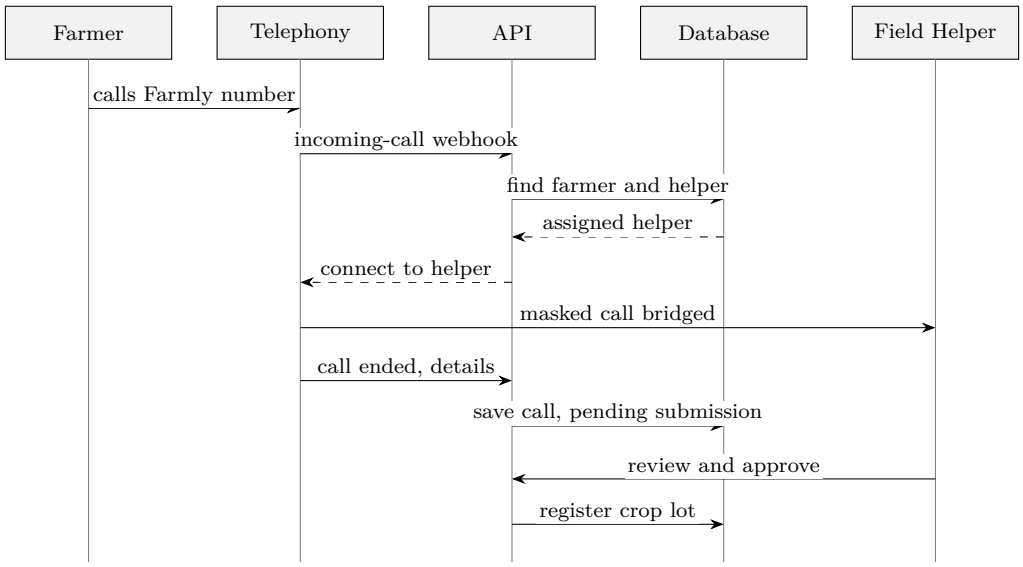


Figure 4.8: Sequence diagram – helper calling (in progress)

Table 4.2: Certificate and traceability API endpoints

Endpoint	**Access**	**Purpose**
POST /certificates/origin	Farmer, helper	Upload an origin certificate
GET /certificates/:id	Public / signed in	Certificate detail and history
GET /certificates/:id/document	Public / signed in	Document link (signed URL if private)
POST /certificates/:id/verify	Public	Run the four verification checks
POST /certificates/:id/retry	Holder	Resume pending storage or registration
POST /certificates/:id/revoke	Holder	Revoke with a reason
POST /purchases	Distributor, retailer	Purchase with an idempotency key
GET /traceability/batches/:id	Signed in	Full chain for a batch
GET /traceability/public/:id	Public	Public-safe product journey
GET /traceability/qr/:id	Public	QR code image

mode, and layouts adapt from mobile to desktop widths.

4.3 Implementation Plan for Next Semester

The next semester extends the platform as shown in Table 4.3.

Table 4.3: Implementation plan for the next semester

Module	**Planned work**
Production deployment	Configure the hosted database, deploy the contract to the Sepolia testnet, deploy the frontend and API, and run the full farmer-to-retailer workflow end to end.
Payments	Link confirmed purchases to the payment tracking workflow (due, initiated, reference submitted, received).
Issuer verification	Allow trusted organisations to confirm the issuer of an uploaded certificate.
AgroSathi voice assistant	Build on the telephony layer of the helper calling module to replace the current rule-based assistant with a multilingual voice interface for farmers.
Fair-price engine	Use live market price data instead of the static benchmark table.
Allied farming	Extend the asset model to dairy, poultry or fisheries batches.
AI and IoT	Evaluate AI-assisted quality observations and sensor data (for example temperature) as additional evidence.
Security and scale	Rate limiting, removal of the development database policy, caching and load testing.

Chapter 5

Results and Discussion

This chapter describes the implementation of Farmly, presents the results of testing and compares the outcome with the existing solutions discussed in Chapter 2.

5.1 Implementation Details

The system was implemented as three packages in one repository: the web application (`frontend`), the REST API (`backend`) and the smart contract tooling (`contracts`), together with the database schema and migration scripts (`database`). The web application contains 56 page routes and the API exposes 62 route handlers.

Authentication and roles

Users register and sign in through the web application, which calls the API and Supabase Authentication. The role selector offers exactly four roles: Farmer, Field Helper, Distributor and Retailer (Figure 5.1). The API rejects any other role, and database triggers ensure that a client can neither create a profile with an unsupported role nor change an existing role. Accounts created earlier with other roles are kept unchanged. Demonstration accounts for the four roles are available in development and are disabled by default in production.

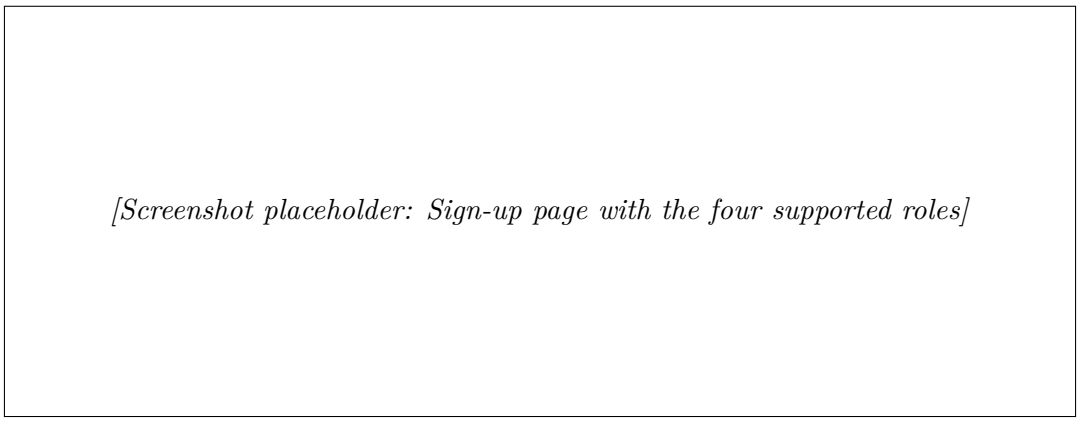


Figure 5.1: Sign-up page with the four supported roles

Dashboards

Each role has its own dashboard built from real records. The farmer dashboard (Figure 5.2) shows the number of registered lots, the available quantity and the number of lots with registered certificates, followed by the farmer's lots with their certificate status and recent sales. The distributor and retailer dashboards show available lots, stock on hand, quantities received through purchases and the status of each transfer certificate. The field helper dashboard shows farmers assisted, pending registration and evidence tasks, and recent field activity. When data is unavailable the dashboards display an explicit empty or unavailable state rather than placeholder figures.

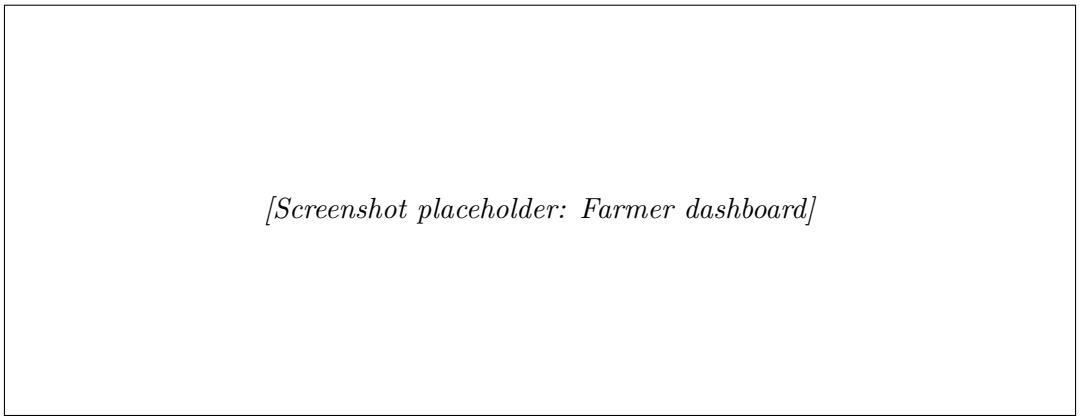


Figure 5.2: Farmer dashboard

Crop lot registration and origin certificate

A farmer, or a field helper on the farmer's behalf, registers a crop lot with its crop, variety, quantity, unit, harvest date and location. The farmer then uploads the supporting document, selecting its type, the issuing organisation as printed, issue and expiry dates, and whether the document may be shown publicly (Figure 5.3). The API checks the file type, size and file signature, computes the SHA-256 hash, uploads the document to IPFS through Pinata, saves the record and registers it on the blockchain. The form states clearly that Farmly records the claims as provided and does not itself verify the issuer.

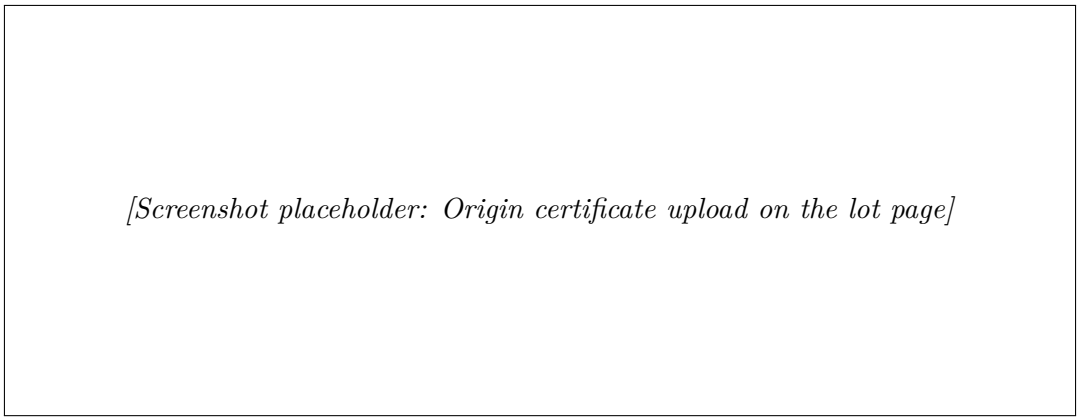


Figure 5.3: Origin certificate upload on the lot page

Marketplace and inspection

The marketplace lists the batches that the signed-in user may buy, each with its available quantity, seller and certificate status, and offers a single *Inspect* action (Figure 5.4). The inspection view (Figure 5.5) shows the batch details and lineage, the complete certificate chain with the farmer's origin certificate first, and previous transfers. The purchase panel is shown only to a distributor inspecting a farmer's lot or a retailer inspecting a distributor's stock, and requires the buyer to confirm that the certificates have been inspected.

Verification and public traceability

Every certificate card offers *Verify certificate* and *Compare a file*. The result shows the overall status and the four individual checks (Figure 5.6). Every

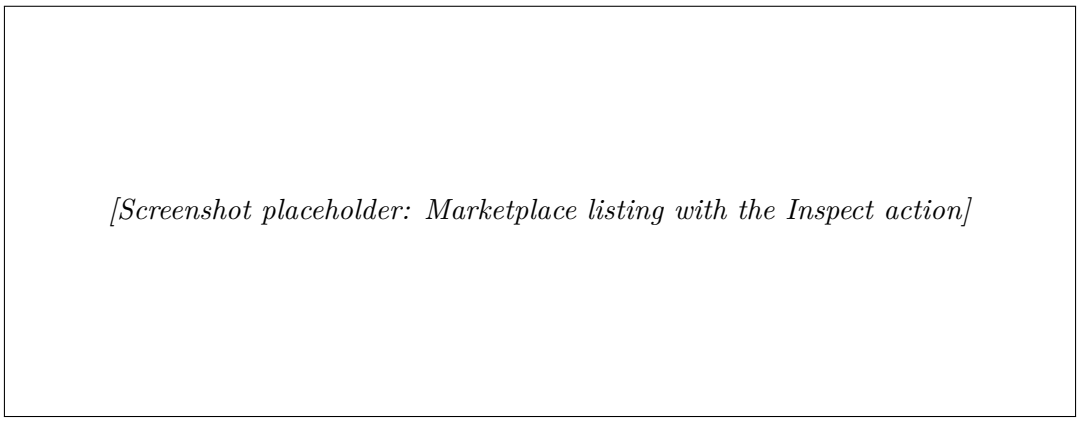


Figure 5.4: Marketplace listing with the Inspect action

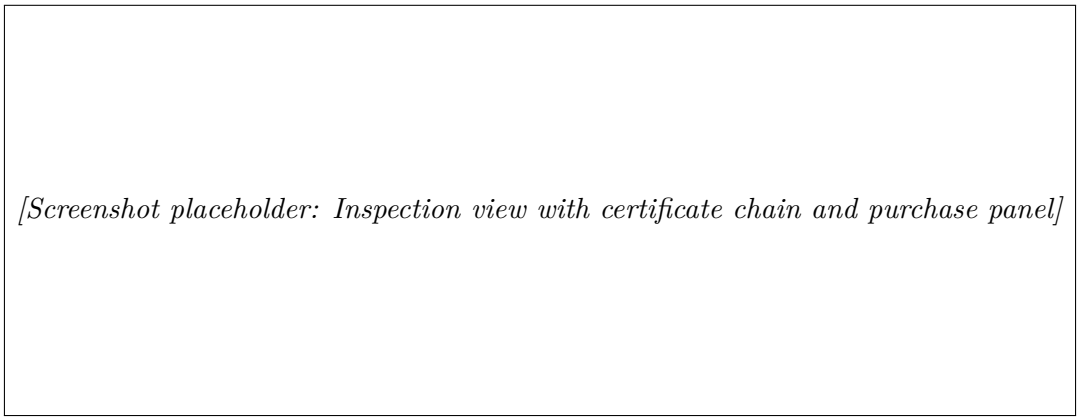


Figure 5.5: Inspection view with certificate chain and purchase panel

batch and certificate also has a QR code that opens the public traceability page (Figure 5.7), which lists the journey in chronological order, marks the farmer's origin certificate separately from transfer certificates, shows warnings for revoked, expired or pending certificates, and explains that a transfer certificate proves a recorded transfer rather than the quality of the produce.



Figure 5.6: Certificate verification result with four separate checks

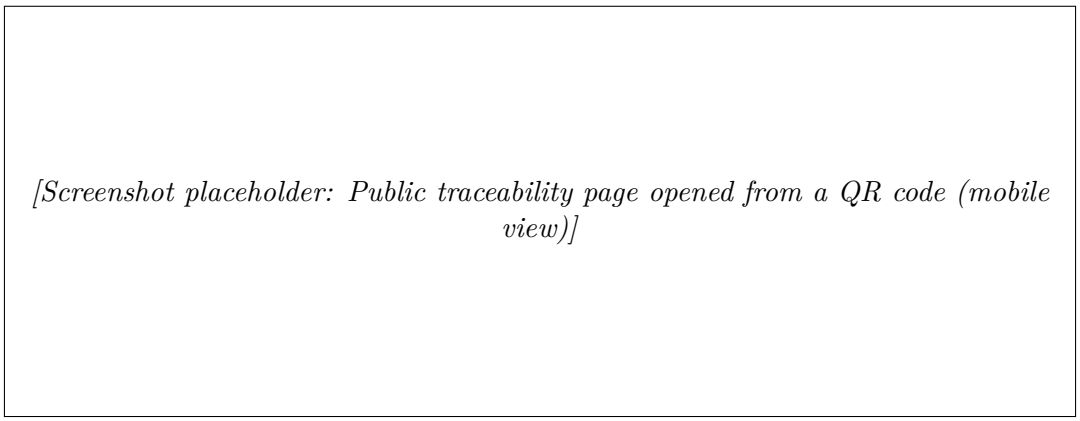


Figure 5.7: Public traceability page opened from a QR code (mobile view)

Theme and accessibility

The light and dark themes are controlled by a single theme provider. The selected theme is stored in the browser, applied before the first paint to avoid a flash of the wrong theme, and switched immediately without reloading the page. Colours that previously worked only on a dark background were replaced by theme-aware colours across the application (Figure 5.8).

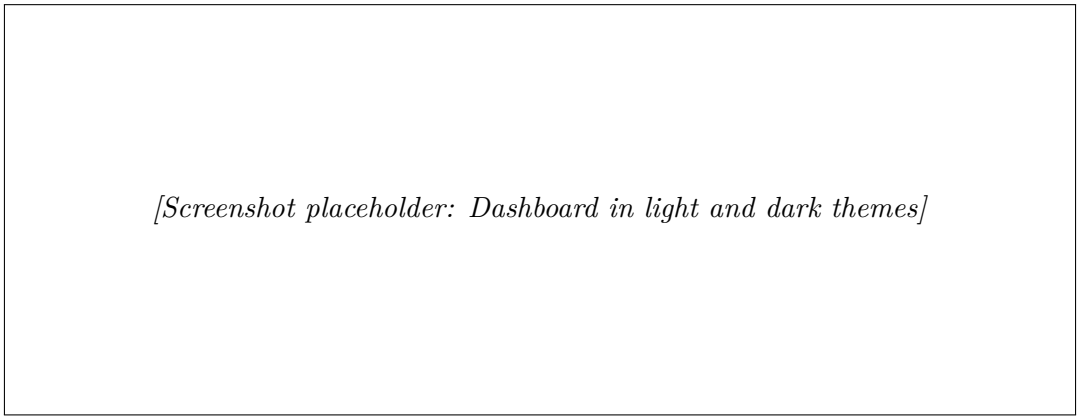


Figure 5.8: Dashboard in light and dark themes

Helper calling (in progress)

The helper calling module described in Chapter 3 is being implemented during this semester. Its database tables for call records and phone submissions are already part of the schema. The remaining work is the integration with the Twilio voice service, the webhook endpoints that receive call events, the *Call my helper* action in the farmer's interface, and the review screen on which a

field helper approves or rejects crop details given by phone. Because the module is not yet complete, it has not been tested and no results are reported for it in this chapter.

5.2 Expected Results

The system was evaluated using the automated tests described in Section 3.4. Table 5.1 summarises the results. All counts refer to tests that were executed; none of the figures are estimates.

Table 5.1: Summary of test results

Test suite	**What was tested**	**Result**
Smart contract	Registration and read-back, source linking, rejection of overwrites, registrar authorisation, input validation, revocation	8 / 8 passed
Database	Schema and migration applied twice, role enforcement, the 500 / 250 / 200 kg scenario, idempotent retries, overselling, wrong-role and self-purchase, revoked source certificate, append-only rules, row-level security	38 / 38 passed
Backend with local blockchain	Registration with confirmed transaction, hash and CID read-back, rejected overwrite, downstream reference, revocation, behaviour without IPFS configuration	11 / 11 passed
IPFS (Pinata)	Public upload and download with matching SHA-256; private upload, refusal through the public gateway, download through a signed URL with matching hash	All passed
API access control	Missing idempotency key, purchase by a farmer, legacy-role demonstration login and registration, legacy demonstration token	Rejected as expected
Interface	Text contrast on 29 pages for all roles in both themes, layout at 375 px width, theme switching and persistence, sign-in redirect for each role	No issues remaining
Builds	Type checking, linting and production builds of the web application and API	Passed
Helper calling	Call routing, number masking, call logging and helper review of phone submissions	Not yet tested (in progress)

Partial purchase scenario

The database tests reproduced the scenario used throughout this report. A farmer registered 500 kg of wheat. A distributor bought 250 kg, after which the farmer's batch showed 250 kg available and a 250 kg child batch was created for the distributor. A retailer then bought 200 kg from the distributor's batch, leaving the distributor with 50 kg, and the retailer's batch carried a lineage of three batches back to the farmer. Repeating a purchase with the same idempotency key returned the original purchase without deducting again; reusing the key with different parameters, buying more than the available quantity, buying with a quantity of zero, a retailer buying directly from a farmer and a buyer purchasing their own batch were all rejected with the source quantity unchanged.

Integrity

On the local blockchain, the hash stored for a registered certificate matched the SHA-256 hash of the document exactly. An attempt to register a different hash under the same certificate identifier was rejected by the contract and the original hash remained unchanged. A revoked certificate was marked as revoked while its original proof remained readable. These results show that any modification of a stored document would be detected as a mismatch during verification.

Comparison with existing systems

Table 5.2 compares Farmly with the systems reviewed in Chapter 2.

Discussion

The results show that the core guarantees of the system hold under test: quantities are conserved across partial purchases and retries, certificate records cannot be altered or deleted, and modified documents are detected. Building the purchase as a single database transaction, before any external service is called, was the key design decision: it keeps the quantity correct even when IPFS or the blockchain fails, and the certificate can then be completed later by a safe retry.

Table 5.2: Comparison of Farmly with existing systems

Feature	**e-NAM**	**Research prototypes**	**IBM Food Trust**	**Farmly**
Marketplace for farmers	Yes	Rarely	No	Yes
Traceability after sale	No	Yes	Yes	Yes
Partial purchases with quantity conservation	No	Rarely	Not stated	Yes
Documents on IPFS, hashes on blockchain	No	Some [5]	No	Yes
Separate integrity, issuer and validity checks	No	No	Not stated	Yes
Price visibility limited to transaction parties	Not stated	No	Not stated	Yes
Public consumer verification by QR	No	Some	Some deployments	Yes
Multilingual, assisted workflows	Partly	No	No	Yes

The main limitation of this phase is that the complete workflow through the user interface, using the hosted database, has not yet been executed, because the production database was being reconfigured at the time of writing. All components of that workflow have been tested individually, as described above. In addition, simultaneous purchases from the same batch are protected by row locking in the database, but were not tested with two concurrent connections, and purchases are not yet linked to the payment tracking workflow. The helper calling module is still being implemented and will be evaluated once it is complete.

Chapter 6

Conclusion and Future Work

The purpose of this project was to give agricultural produce a verifiable digital record that follows it from the farmer to the retailer, so that buyers can inspect where a product came from and consumers can verify it, without exposing confidential prices or personal information.

In this phase we built the core of that system. Farmly supports four roles, enforced in the API and in the database. Farmers register crop lots and attach certificates; each certificate document is hashed with SHA-256, stored on IPFS and anchored on a blockchain through a smart contract that does not allow records to be overwritten. Distributors and retailers inspect the full certificate history before purchasing, and every purchase, including a partial purchase, is processed atomically, creates a child batch and generates a new certificate linked to all upstream certificates. Consumers can scan a QR code to view the product's journey and verify each certificate, with integrity, registration, issuer authenticity and validity reported separately. Automated tests of the smart contract, the database, the blockchain integration and the IPFS storage all pass, and the interface works in both light and dark themes on mobile and desktop screens. The helper calling module, which lets farmers reach their field helper by telephone, has been designed and its database tables are in place; its implementation is being completed in this semester.

The most important lesson of the project was that a blockchain proves only that a record has not changed since it was written; it does not prove that the record was true. For this reason the system never labels a certificate as verified merely because a file was uploaded or a transaction was submitted, and it states clearly which claims have been checked and which have not. A second lesson

was that reliability depends on ordering: reserving the quantity in the database first, and treating IPFS and blockchain steps as retryable, prevents both lost purchases and duplicate certificates.

Future Work

- Complete and test the helper calling module: telephony integration, number masking, call logging and the helper review of crop details given by phone.
- Deploy the database, API and web application, deploy the smart contract to the Sepolia testnet, and execute the complete farmer-to-retailer workflow end to end.
- Integrate purchases with payment tracking, and test concurrent purchases from the same batch.
- Allow trusted organisations such as certification bodies or laboratories to verify certificate issuers.
- Develop the AgroSathi voice assistant for farmers in local languages, and a fair-price engine based on live market data.
- Extend the asset model to allied farming domains such as dairy, poultry and fisheries.
- Evaluate AI-assisted quality observations and IoT sensor data as additional evidence, and strengthen security with rate limiting and removal of the development database policy.

References

[1] Government of India, Ministry of Agriculture & Farmers Welfare, "National Agriculture Market (e-NAM)," https://enam.gov.in.

[2] F. Tian, "An agri-food supply chain traceability system for China based on RFID & blockchain technology," in *Proc. 13th International Conference on Service Systems and Service Management (ICSSSM)*, 2016, pp. 1–6.

[3] F. Tian, "A supply chain traceability system for food safety based on HACCP, blockchain & Internet of Things," in *Proc. International Conference on Service Systems and Service Management (ICSSSM)*, 2017, pp. 1–6.

[4] M. P. Caro, M. S. Ali, M. Vecchio and R. Giaffreda, "Blockchain-based traceability in agri-food supply chain management: A practical implementation," in *Proc. IoT Vertical and Topical Summit on Agriculture – Tuscany (IOT Tuscany)*, 2018, pp. 1–4.

[5] K. Salah, N. Nizamuddin, R. Jayaraman and M. Omar, "Blockchain-based soybean traceability in agricultural supply chain," *IEEE Access*, vol. 7, pp. 73295–73305, 2019.

[6] J. Lin, Z. Shen, A. Zhang and Y. Chai, "Blockchain and IoT based food traceability for smart agriculture," in *Proc. 3rd International Conference on Crowd Science and Engineering (ICCSE)*, 2018.

[7] K. Leng, Y. Bi, L. Jing, H.-C. Fu and I. Van Nieuwenhuyse, "Research on agricultural supply chain system with double chain architecture based on blockchain technology," *Future Generation Computer Systems*, vol. 86, pp. 641–649, 2018.

[8] IBM, "IBM Food Trust: A new era for the world's food supply," https://www.ibm.com/products/supply-chain-intelligence-suite/food-trust.

[9] A. Kamilaris, A. Fonts and F. X. Prenafeta-Boldú, "The rise of blockchain technology in agriculture and food supply chains," *Trends in Food Science & Technology*, vol. 91, pp. 640–652, 2019.

[10] J. F. Galvez, J. C. Mejuto and J. Simal-Gandara, "Future challenges on the use of blockchain for food traceability analysis," *TrAC Trends in Analytical Chemistry*, vol. 107, pp. 222–232, 2018.

[11] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008. https://bitcoin.org/bitcoin.pdf.

[12] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," Ethereum Project Yellow Paper, 2014.

[13] J. Benet, "IPFS – Content addressed, versioned, P2P file system," arXiv:1407.3561, 2014.

[14] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, 2015.

[15] OWASP Foundation, "OWASP API Security Top 10 – 2023," https://owasp.org/API-Security/.

[16] Vercel, "Next.js Documentation," https://nextjs.org/docs.

[17] Supabase, "Supabase Documentation: Row Level Security," https://supabase.com/docs/guides/database/postgres/row-level-security.

[18] Pinata, "Pinata API Reference (Files API v3)," https://docs.pinata.cloud.

[19] Nomic Foundation, "Hardhat Documentation," https://hardhat.org/docs.

[20] R. Moore, "ethers.js v6 Documentation," https://docs.ethers.org/v6/.

[21] Twilio, "Programmable Voice Documentation," https://www.twilio.com/docs/voice.

Appendix A

Plagiarism Report

Appendix B

List of Publications

ACKNOWLEDGMENT

We take this opportunity to express our profound gratitude and deep regards to our guide and Project Coordinator **Prof. Shahegul Afroz**, Department of Computer Science & Engineering (Data Science), for the exemplary guidance, monitoring and constant encouragement throughout the completion of this report. We are truly grateful for the efforts made to improve our understanding of the various concepts and technical skills required in our project. The blessing, help and guidance given from time to time shall carry us a long way in the journey of life on which we are about to embark. We take this privilege to express our sincere thanks to **Dr. Ramjan A. Khatik**, Director, AIKTC and **Dr. Rajendra Magar**, Dean, SoET for providing the much necessary facilities. We are also thankful to **Prof. Zeeshan Khan**, Head of Department of Computer Science & Engineering (Data Science), AIKTC, New Panvel, Navi Mumbai for the generous support. Last but not the least we would also like to thank all those who have directly or indirectly helped us in completion of this thesis.

Aafia Shaikh

Talha Siddiqui